

Python: estructuras de datos

Qué crear, qué guardar y cuánto cuesta. Aplicación a grafos en la cara 2.

Notación: n = elementos del contenedor; V = nodos; E = aristas; $g(u)$ = vecinos de u . **$O(1)$** constante; **$O(\log n)$** logarítmico; **$O(n)$** recorre n ; **$O(n^2)$** cuadrático. La complejidad describe cómo crece el trabajo, no segundos exactos.

Estructura y creación	Qué conserva	Operaciones y coste	Cuándo elegirla
list <code>a = []</code> <code>a = [1, 2]</code>	Orden y repetidos. Mutable: se modifica.	<code>a[i], a[i] = x</code> : $O(1)$ <code>append(x), pop()</code> : $O(1)^*$ <code>x in a, pop(0), remove(x)</code> : $O(n)$	Camino y orden de visita. Pila DFS: último en entrar, primero en salir.
tuple <code>t = (u, v)</code> Un elemento: <code>(u,)</code>	Orden y repetidos. Inmutable: no se pueden sustituir sus elementos.	<code>t[i]</code> : $O(1)$ <code>x in t</code> : $O(n)$ No tiene <code>append</code> ni <code>pop</code> .	Coordenadas, aristas, registros y claves compuestas (origen, destino).
set <code>s = set()</code> <code>s = {u, v}</code>	Elementos únicos. Sin orden de recorrido garantizado. Mutable.	<code>x in s, add(x), discard(x)</code> : $O(1)(P)$ No se accede con <code>s[i]</code> .	Visitados y eliminar duplicados. <code>discard</code> no falla si no existe.
dict <code>d = {}</code> <code>d = {u: 0}</code>	Clave $\rightarrow$ valor. Claves únicas; orden de inserción. Mutable.	<code>d[u], d.get(u, defecto), u in d, d[u] = x, del d[u]</code> : $O(1)(P)$	Distancias, padres, penalizaciones y listas de adyacencia.
deque <code>q = deque([u])</code> (ver import en BFS)	Orden y repetidos. Mutable. Cola: primero en entrar, primero en salir.	<code>append(x), appendleft(x), pop(), popleft()</code> : $O(1)$ Buscar o acceder al centro: $O(n)$.	Cola BFS. Extraer por delante sin desplazar el resto.
heapq sobre una list <code>import heapq</code> <code>h = []</code>	Montículo mínimo: la menor prioridad en <code>h[0]</code> . El resto no está ordenado.	<code>heapq.heappush(h, x)</code> : $O(\log n)$ <code>heapq.heappop(h)</code> : $O(\log n)$ <code>h[0]</code> : $O(1)$ <code>heapq.heapify(h)</code> : $O(n)$.	Cola de prioridad de Dijkstra. No usar <code>pop(0)</code> sobre el heap.

* $O(1)$ **amortizado**: alguna operación puede redimensionar en $O(n)$, pero una secuencia sale constante por operación. (P) $O(1)$ **promedio** con hashing; peor caso $O(n)$. Se asumen claves de coste constante. Costes habituales de CPython.

Crear, consultar y copiar

```
a = [v for v in G]      # list
s = {v for v in G}      # set
d = {v: 0 for v in G}    # dict
for v, coste in d.items():
    pass                 # clave y valor
```

Consulta: `x in d` busca claves; buscar en `d.values()` cuesta $O(n)$. `d[u]` falla si falta u ; `d.get(u, 0)` devuelve 0 sin insertar. `len(...)`: $O(1)$; recorrer: $O(n)$.

Listas: `append(x)` añade x ; `extend(it)` añade k elementos: $O(k)$. Slice `a[i:j]`: copia k , $O(k)$. `sorted(a)` crea una lista; `a.sort()` la modifica: $O(n \log n)$. `append` y `sort` devuelven `None`.

Copias: `b = a` comparte objeto. `a.copy()` copia solo el exterior en $O(n)$; comparte objetos anidados. Crea filas con `[[] for _ in range(V)]`, nunca `[[[]] * V]`.

Hashable: claves de dict y elementos de set: int, str o tuplas de elementos hashables; no list/dict. `frozenset(s)`: conjunto inmutable.

Guardar en disco (si necesitas conservar datos)

JSON sirve para listas y diccionarios sencillos: `json.dump(datos, f)` guarda; `json.load(f)` carga (con `import json` y un archivo abierto mediante `with open(...)`). Convierte set/deque a list; las tuplas se cargan como listas y las claves int de dict como str. No guarda directamente un `nx.Graph`.

Representar un grafo

```
adj = {0: [1, 2], 1: [0], 2: [0]}
vecinos = adj[0]
# Pesos por arista, grafo dirigido:
pesos = {0: {1: 4, 2: 7}, 1: {}, 2: {}}
```

Adyacencia usa $O(V+E)$ de memoria. Con `dict[u] = list` recorrer vecinos cuesta $O(g(u))$, y comprobar `v in adj[u]` también. Con `dict[u] = set` o dict de pesos, comprobar una arista cuesta $O(1)$ promedio. En un grafo no dirigido, guarda cada arista en ambos sentidos.

Matriz: `M[u][v]` consulta en $O(1)$; memoria $O(V^2)$, recorrer vecinos $O(V)$. Para grafos densos; en laberintos suele convenir adyacencia.

En tu práctica: `G = maze.maze_graph()` ya es un `nx.Graph`. `G.neighbors(u)` itera sus vecinos; `G[u][v]` contiene atributos de la arista. No necesitas convertirlo a otra representación.

Nodos 0..V-1: `dist = [float("inf")] * V, seen = [False] * V`: acceso $O(1)$ y menos memoria habitual que dict/set. Con etiquetas arbitrarias usa dict/set. Contenedores: $O(n)$ espacio, sin contar objetos anidados.

Grafos: usar la estructura adecuada

Plantillas con NetworkX. Suponen que los nodos de entrada existen en G.

BFS: cola deque, mínimo nº de aristas si todos los pasos cuestan igual. **DFS:** pila o recursión, explora a fondo; no garantiza camino mínimo. **Dijkstra:** heap, mínimo coste con pesos **no negativos**. En BFS/DFS: set para pertenencia + list si necesitas conservar el orden.

BFS: cola + set + list

```
from collections import deque
def bfs(G, origin):
    vistos = {origin}
    q = deque([origin])
    orden = []
    while q:
        u = q.popleft()
        orden.append(u)
        for v in G.neighbors(u):
            if v not in vistos:
                vistos.add(v) # al encolar
                q.append(v)
    return orden
```

En LAB01 se pide el **recorrido de los alcanzables**, incluido origin, no un camino hasta un destino. Marca al encolar: evita introducir el mismo nodo varias veces. El orden entre vecinos depende del orden en G.

DFS: recursión (pila de llamadas)

```
def recorrido_dfs(G, origin):
    vistos, orden = set(), []
    def visita(u):
        vistos.add(u)
        orden.append(u)
        for v in G.neighbors(u):
            if v not in vistos:
                visita(v)
    visita(origin)
    return orden
```

En profundidad grande, la recursión puede dar RecursionError. La versión iterativa usa una list como pila: append y pop(). El orden exacto puede cambiar respecto al recursivo.

BFS/DFS con adyacencia: tiempo $O(V+E)$, memoria auxiliar $O(V)$. Desde un origen solo recorren su parte alcanzable. Usar una list para `v in vistos` o `pop()` puede elevar el tiempo a cuadrático. Con matriz, el recorrido completo cuesta $O(V^2)$.

Aplicarlo a holes y checkpoints

holes: usa `coste = lambda u, v: holes_dct.get(v, 1)`. La penalización **sustituye** al coste 1. Esta guía paga cada entrada en v (incluido el destino) y deja el origen gratis: sin penalizaciones, `cost = len(path) - 1`; si `origin = destino`, `([origin], 0)`. **El enunciado no aclara el coste del origen**; si se cobra, inicia dist y heap con su coste.

checkpoint: resuelve `origen → extra y extra → destino`. Une `p1 + p2[1:]` y suma `c1 + c2` con origen gratis en cada tramo. Si cada tramo cobra su origen, descuenta el coste de extra una vez. Dos Dijkstra siguen costando $O(T)$, más $O(L)$ para unir el resultado.

k extras: `itertools.permutations(extras)` genera $k!$ órdenes (tuplas). Recalcular todos los tramos: $O(k! \cdot (k+1) \cdot T)$. Mejor: hasta $k+1$ Dijkstra **completos**, desde origen y cada extra, sin parar en un destino; guardar distancias/padres y evaluar cada orden en $O(k)$. Total $O((k+1)T + k^2 + k \cdot k! + L)$. Reconstruye solo el ganador; une tramos con `ruta.extend(tramo[1:])`.

Cuidado: cache `(u, v)` solo para el mismo grafo y penalizaciones: el coste inverso puede diferir. Se permiten revisitas. Sin camino: esta plantilla devuelve `([], inf)`. Respeta cabeceras y retornos; no modifiques `holes_dct={}`. En código propio usa None por defecto.

Fuentes: cuaderno LAB01; documentación oficial de Python: estructuras, costes, heapq, JSON; NetworkX: vecinos.

Dijkstra: dist + padre + heap

```
from heapq import heappush, heappop
from itertools import count
def dijkstra(G, s, t, coste):
    inf = float("inf")
    turno = count()
    dist, padre = {s: 0}, {s: None}
    h = [(0, next(turno), s)]
    while h:
        du, _, u = heappop(h)
        if du != dist[u]:
            continue # entrada antigua
        if u == t:
            camino = []
            while u is not None:
                camino.append(u)
                u = padre[u]
            camino.reverse()
            return camino, du
        for v in G.neighbors(u):
            alt = du + coste(u, v)
            if alt < dist.get(v, inf):
                dist[v], padre[v] = alt, u
                heappush(h, (alt, next(turno), v))
    return [], inf
```

`count()` desempata sin comparar los nodos. Si mejora una distancia, inserta otra entrada y descarta la vieja al extraerla.

No marques al insertar; el destino queda resuelto al extraer su entrada vigente.

Pesos de arista: pasa como coste una función lambda `u, v: G[u][v].get("weight", 1)`. Guarda solo `padre[v] = u`; copiar caminos enteros al explorar consume más tiempo y memoria.

Este heap con duplicados: $T = O((V+E) \log(V+1))$ en grafos simples; memoria auxiliar $O(V+E)$. Más generalmente, el heap puede tener $O(E)$ entradas. Reconstruir un camino de L nodos cuesta $O(L)$.